

Conjunto de Perguntas para Avaliação de um Sistema de Recuperação de Contexto

1 Contexto

Este entregável apresenta um conjunto de 30 perguntas destinadas à avaliação de um protótipo de recuperação de contexto para aplicações de Inteligência Artificial Generativa.

O corpus utilizado no experimento é o livro *Pense Python*, 3ª edição, de Allen B. Downey, em sua versão em língua portuguesa. O sistema de recuperação recebe uma consulta em linguagem natural e retorna os k trechos, ou *chunks*, considerados mais relevantes.

A recuperação implementada nesta etapa é lexical e utiliza uma representação baseada em TF-IDF (*Term Frequency–Inverse Document Frequency*). O TF-IDF atribui maior peso aos termos que são frequentes em um determinado documento ou *chunk*, mas menos frequentes no conjunto total de documentos.

De forma geral, o peso de um termo t em um documento d pode ser representado como:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \text{idf}(t)$$

em que $\text{tf}(t, d)$ representa a frequência do termo t no documento d , enquanto $\text{idf}(t)$ representa sua frequência inversa no conjunto de documentos.

O pré-processamento realizado antes da recuperação inclui:

1. conversão do texto para letras minúsculas;
2. remoção de acentos;
3. substituição de caracteres não alfanuméricos por espaços;
4. tokenização do texto.

As perguntas foram definidas antes da execução do sistema de recuperação, evitando que os resultados retornados pelo sistema influenciassem a formulação das consultas utilizadas na avaliação.

2 Critérios de Construção

O conjunto foi organizado em três níveis de dificuldade:

- **10 perguntas fáceis:** apresentam forte correspondência lexical com a terminologia utilizada no corpus;
- **10 perguntas médias:** apresentam alguma variação de vocabulário, mantendo termos importantes em comum com o conteúdo relevante;
- **10 perguntas difíceis:** apresentam formulações mais naturais ou indiretas, reduzindo a correspondência lexical direta com os trechos esperados.

As perguntas foram construídas exclusivamente a partir de conceitos presentes na 3ª edição em português do livro e distribuídas por diferentes capítulos e tópicos, incluindo operadores, expressões, variáveis, funções, condicionais, recursão, iteração, strings, listas, dicionários, tuplas, arquivos, classes, objetos, métodos, herança e conjuntos.

A divisão por dificuldade tem como objetivo permitir a análise do comportamento do mecanismo de recuperação diante de diferentes níveis de proximidade lexical entre a consulta e o corpus.

3 Conjunto de Perguntas

ID	Pergunta	Dificuldade	Tema principal	Palavras-chave esperadas	Capítulo/seção provável
Q01	Qual é a diferença entre divisão convencional e divisão inteira em Python?	Fácil	Operadores aritméticos	divisão, divisão inteira, operador, inteiro	Cap. 1 – Operadores aritméticos
Q02	Como os parênteses alteram a ordem de operações em uma expressão?	Fácil	Expressões	expressão, parênteses, ordem, operações	Cap. 1 – Expressões
Q03	Como criar uma variável e atribuir um valor a ela em Python?	Fácil	Variáveis e atribuição	variável, valor, atribuição, expressão	Cap. 2 – Variáveis
Q04	Quais nomes podem ou não ser usados como nomes de variáveis em Python?	Fácil	Nomes de variáveis	nomes, variáveis, palavra-chave, sintaxe	Cap. 2 – Nomes de variáveis
Q05	Como definir uma nova função com <code>def</code> e depois chamá-la?	Fácil	Definição de funções	função, <code>def</code> , definição, chamada	Cap. 3 – Definindo novas funções
Q06	Qual é a relação entre um argumento passado para uma função e seu parâmetro?	Fácil	Parâmetros e argumentos	argumento, parâmetro, função, valor	Cap. 3 – Parâmetros
Q07	O que significa encapsular um código em uma função e como um parâmetro pode generalizar essa função?	Fácil	Encapsulamento e generalização	encapsulamento, generalização, função, parâmetro	Cap. 4 – Encapsulamento e generalização
Q08	Como usar <code>if</code> , <code>elif</code> e <code>else</code> para tratar diferentes condições em um programa?	Fácil	Condicionais	<code>if</code> , <code>elif</code> , <code>else</code> , condição	Cap. 5 – Instruções <code>if</code> / A cláusula <code>else</code> / Condicionais encadeadas
Q09	O que é recursão e como uma função pode chamar a si mesma?	Fácil	Recursão	recursão, função, chamada, recursiva	Cap. 5 – Recursão
Q10	Como uma função usa <code>return</code> para devolver um valor?	Fácil	Valores devolvidos	<code>return</code> , valor devolvido, função, variável	Cap. 6 – Algumas funções devolvem valores
Q11	Como usar um laço <code>for</code> para percorrer os caracteres de uma string?	Média	Iteração em strings	<code>for</code> , string, caracteres, variável de iteração	Cap. 7 – Laços de repetição e strings
Q12	Como ler um arquivo de palavras linha por linha e remover a quebra de linha de cada palavra?	Média	Leitura de arquivos	<code>open</code> , <code>readline</code> , <code>strip</code> , arquivo	Cap. 7 – Lendo a lista de palavras
Q13	Como atualizar uma variável dentro de um laço para contar ocorrências?	Média	Atualização de variáveis	atualização, variável, incremento, laço	Cap. 7 – Atualizando variáveis
Q14	Como acessar o último caractere de uma string usando um índice negativo?	Média	Índices em strings	string, índice, negativo, caractere	Cap. 8 – Uma string é uma sequência

ID	Pergunta	Dificuldade	Tema principal	Palavras-chave esperadas	Capítulo/seção provável
Q15	Por que não é possível alterar diretamente um caractere de uma string em Python?	Média	Imutabilidade de strings	string, imutável, TypeError, caractere	Cap. 8 – Strings são imutáveis
Q16	Como <code>append</code> e <code>extend</code> alteram uma lista de maneiras diferentes?	Média	Métodos de listas	append, extend, lista, elementos	Cap. 9 – Métodos de listas
Q17	Por que duas variáveis podem acabar se referindo ao mesmo objeto lista?	Média	Referências e objetos	variável, lista, objeto, referência	Cap. 9 – Listas
Q18	Como um dicionário associa uma chave a um valor e como recuperar esse valor?	Média	Mapeamento com dicionários	dicionário, chave, valor, mapeamento	Cap. 10 – Um dicionário é um mapeamento
Q19	Por que verificar a existência de uma chave em um dicionário pode ser muito mais rápido do que procurar um elemento em uma lista?	Média	Eficiência de busca	dicionário, lista, in, tabela hash	Cap. 10 – O operador in
Q20	Como a atribuição de tupla permite trocar os valores de duas variáveis sem usar uma variável temporária?	Média	Atribuição de tuplas	tupla, atribuição, variáveis, temporária	Cap. 11 – Atribuição de tupla
Q21	Se eu fornecer apenas o nome <code>memo.txt</code> , como o Python decide em qual diretório procurar o arquivo e como obtenho seu caminho completo?	Difícil	Caminhos de arquivos	diretório atual, caminho relativo, caminho absoluto, abspath	Cap. 13 – Nomes de arquivos e caminhos
Q22	Por que modificar uma lista recuperada de uma prateleira não atualiza automaticamente o valor armazenado nela?	Difícil	Persistência de estruturas de dados	prateleira, lista, chave, atualizar, shelve	Cap. 13 – Armazenando estruturas de dados
Q23	Como criar um tipo próprio para representar uma hora do dia e armazenar hora, minuto e segundo dentro de cada objeto?	Difícil	Classes e atributos	classe, Time, objeto, atributos, instanciación	Cap. 14 – Tipos definidos pelo programador / Atributos
Q24	Quando uma função recebe um objeto <code>Time</code> e altera seus atributos, por que o objeto original também é modificado?	Difícil	Mutabilidade de objetos	objeto, Time, mutável, atributos, modificar	Cap. 14 – Objetos são mutáveis
Q25	Por que <code>start.print_time()</code> consegue usar <code>start</code> dentro do método mesmo sem ele aparecer explicitamente entre os argumentos da chamada?	Difícil	Métodos e self	método, self, objeto, receptor, argumento	Cap. 15 – Definindo métodos

ID	Pergunta	Dificuldade	Tema principal	Palavras-chave esperadas	Capítulo/seção provável
Q26	O que precisa ser definido em Time para que dois objetos possam ser somados usando diretamente o operador + ?	Difícil	Sobrecarga de operador	<code>__add__</code> , operador, Time , método especial, sobrecarga	Cap. 15 – Sobrecarga de operador
Q27	Por que copiar um Rectangle pode fazer o original e a cópia continuarem compartilhando o mesmo objeto Point ?	Difícil	Cópia rasa e profunda	<code>copy</code> , <code>deepcopy</code> , Rectangle , Point , cópia profunda	Cap. 16 – Cópia profunda
Q28	Como dois objetos diferentes podem representar o mesmo ponto sem serem o mesmo objeto na memória?	Difícil	Identidade e equivalência	identidade, equivalência, objeto, <code>is</code> , <code>==</code>	Cap. 16 – Equivalência e identidade
Q29	Como criar uma classe mais específica que reutiliza atributos e métodos de outra classe, mas acrescenta comportamento próprio?	Difícil	Herança e especialização	herança, classe mãe, classe filha, especialização	Cap. 17 – Herança / Especialização
Q30	Como eliminar valores repetidos de uma sequência sem percorrer manualmente todos os elementos para verificar duplicatas?	Difícil	Conjuntos	<code>set</code> , conjunto, elementos únicos, duplicatas	Cap. 18 – Conjuntos

4 Verificação do Conjunto

4.1 Distribuição por dificuldade

O conjunto possui exatamente 30 perguntas, distribuídas da seguinte forma:

- 10 perguntas fáceis: Q01–Q10;
- 10 perguntas médias: Q11–Q20;
- 10 perguntas difíceis: Q21–Q30.

Essa distribuição permite analisar separadamente o desempenho da recuperação em consultas com diferentes níveis de correspondência vocabular com o corpus.

4.2 Cobertura de capítulos e tópicos

As perguntas cobrem conteúdos dos capítulos 1 a 11 e dos capítulos 13 a 18 da 3ª edição de *Pense Python*.

O Capítulo 12 não foi utilizado diretamente neste conjunto de consultas. A seleção não busca necessariamente produzir o mesmo número de questões para cada capítulo, mas incluir diferentes tipos de conteúdo e diferentes níveis de dificuldade lexical.

Entre os principais tópicos contemplados estão:

- operadores aritméticos e expressões;
- variáveis e atribuição;
- nomes de variáveis e palavras-chave;
- funções;
- parâmetros e argumentos;
- encapsulamento e generalização;
- condicionais;
- recursão;
- valores devolvidos;
- iteração;
- atualização de variáveis;
- leitura de arquivos;
- strings e índices;
- imutabilidade;

- listas e seus métodos;
- referências a objetos;
- dicionários e tabelas hash;
- tuplas;
- caminhos e diretórios;
- persistência de estruturas de dados;
- classes e atributos;
- mutabilidade de objetos;
- métodos e `self`;
- sobrecarga de operadores;
- identidade e equivalência;
- cópia rasa e profunda;
- herança e especialização;
- conjuntos.